DSP. As a general rule, expect that the beginning and ending samples in processed signals will be quite useless.

Now the math. Using the convolution machine as a guideline, we can write the standard equation for convolution. If $x[n]$ is an N point signal running from 0 to N-1, and $h[n]$ is an M point signal running from 0 to M-1, the convolution of the two: $y[n] = x[n] * h[n]$, is an $N+M$-1 point signal running from 0 to $N+M$-2, given by:

EQUATION 6-1
The convolution summation. This is the formal definition of convolution, written in the shorthand: $y[n] = x[n] * h[n]$. In this equation, $h[n]$ is an M point signal with indexes running from 0 to M-1.

$$y[i] = \sum_{j=0}^{M-1} h[j]\, x[i-j]$$

This equation is called the **convolution sum**. It allows each point in the output signal to be calculated independently of all other points in the output signal. The index, i, determines which sample in the output signal is being calculated, and therefore corresponds to the left-right position of the convolution machine. In computer programs performing convolution, a loop makes this index run through each sample in the output signal. To calculate one of the output samples, the index, j, is used *inside* of the convolution machine. As j runs through 0 to M-1, each sample in the impulse response, $h[j]$, is multiplied by the proper sample from the input signal, $x[i-j]$. All these products are added to produce the output sample being calculated. Study Eq. 6-1 until you fully understand how it is implemented by the convolution machine. Much of DSP is based on this equation. (Don't be confused by the n in $y[n] = x[n] * h[n]$. This is merely a place holder to indicate that *some* variable is the index into the array. Sometimes the equations are written: $y[\,] = x[\,] * h[\,]$, just to avoid having to bring in a meaningless symbol).

Table 6-2 shows a program for performing convolutions using the *output side algorithm*, a direct use of Eq. 6-1. This program produces the same output signal as the program for the *input side algorithm*, shown previously in Table 6-1. Notice the main difference between these two programs: the input side algorithm loops through each sample in the *input signal* (line 220 of Table 6-1), while the output side algorithm loops through each sample in the *output signal* (line 180 of Table 6-2).

Here is a detailed operation of this program. The FOR-NEXT loop in lines 180 to 250 steps through each sample in the output signal, using I% as the index. For each of these values, an inner loop, composed of lines 200 to 230, calculates the value of the output sample, Y[I%]. The value of Y[I%] is set to zero in line 190, allowing it to accumulate the products inside of the convolution machine. The FOR-NEXT loop in lines 200 to 240 provide a direct implementation of Eq. 6-1. The index, J%, steps through each